

Monthly outcome: paths, trajectories and behaviour

M16 • 21 Jan 2028 - 15 Feb 2028 • 75 hours

Prepare comparative planner tests and a small autonomous task with a behaviour tree. The robot receives a goal, finds a valid path, builds a speed-limited trajectory along it and monitors execution, explicitly responding when it cannot continue. This month's examples are a body route among obstacles and a manipulator approach to an object. These foundations precede foot-placement planning.

Three different questions

Search determines which states connect the start and goal. A geometric path $q(s)$ describes a configuration sequence without time. A trajectory $q(t)$ adds time, velocity and acceleration. Task logic decides when to plan, wait, cancel and recover. A* success does not mean an actuator can execute the path; successful arm planning does not mean the object has already been grasped.

What to use from earlier months

Use the M15 map with unknown cells, M14 scene geometry, and link kinematics and actuator limits from earlier modules. Check that start and goal use one frame and that angular and linear units are consistent. The prerequisites are reading URDF, running ROS 2 actions, constructing a graph and explaining a derivative. Review how a joint-angle trajectory produces rad/s and rad/s^2 .

Two small experimental setups

Setup A is a 2D room map and a body with a finite planar footprint. You may use a simulated wheeled base to learn Nav2. Setup B is a ready-made manipulator model in MoveIt 2, with a table and a box. Choose one prepared package for each, compatible with Ubuntu 24.04 and ROS 2 Jazzy. Do not build a complex robot model and a new environment from scratch this month.

The final package contains your Dijkstra/A* implementations, comparison of at least two methods, a configuration-sampling experiment, trajectory checks, BT and FSM logs, and recovery scenarios. Foothold planning, hexapod kinodynamics and contact optimisation belong to later specialisation. Explain now why a circular body footprint and a free floor cell do not establish that a particular multilegged step is feasible.

75 hours: from the map to controlled execution

Four work blocks

1-20: graphs, BFS, Dijkstra, A*, heuristics and costmaps; 10 hours of theory and analysis +10 of practice. 21-40: configuration space, collision checking, PRM/RRT/RRT* and the MoveIt scene; 10+10. 41-60: time parameterisation, splines, velocity, acceleration and jerk limits; 10+10. 61-75: FSM, BT, recovery, timeouts and MDP/POMDP foundations; 5+10. The total budget is 35 hours of analysis and 40 hours of practice.

Use 2-hour sessions for the first three theory blocks; divide the last as 2+2+1. Sunday practical blocks of 10 hours include implementation, measurement and reporting; breaks do not count as study hours. The numbering below specifies study-hour order, not calendar dates.

Module-boundary dates may overlap; holidays, rest and separate checkpoints remain in the original calendar without counting hours twice.

Repository structure

Store synthetic and input maps in `maps/`; algorithms and the common interface in `search/`; OMPL settings in `sampling/`; MoveIt objects and robot parameters in `scenes/`; `q,t` files and limits in `trajectories/`; the tree, FSM and adapters in `autonomy/`; and scenarios, seeds and results in `benchmark/`. Tie every result to a map version and body footprint. After an obstacle moves, do not silently continue using the old geometry.

One hypothesis per experiment

To test "A* expands fewer vertices", retain the same graph, cost and tie-breaking order. To test "RRT* improves the path", compare time and cost under the same budget, including failed runs with different seeds. To test "smoothing improves motion", recheck collisions: a spline can cut a corner through a wall. First state the reason for the comparison, then choose the metric.

Finish GUI work by saving a machine-readable scene and launch commands. Do not rely only on dragging a goal manually in RViz. Prepare a small reference example for each task that another person can check without the complete setup: a known optimal path, a segment through an obstacle, a deliberately over-fast trajectory or an expected state trace.

Graph search and the cost of a body route

What to study

BFS finds the shortest path by edge count when edges have equal cost; Dijkstra handles nonnegative costs; A* adds a heuristic estimate h of cost to the goal. For optimality, study admissibility and consistency of h , vertex reopening and termination criteria. Manhattan distance is suitable for a 4-connected grid with unit steps; recheck admissibility for different motions and costs.

A costmap stores assigned movement costs, while an occupancy map stores occupied, free and unknown cells. For a finite-size body, account for its footprint, or inflate obstacles by the circular-model radius with a justified margin. Clearance must account for geometry and uncertainty. Explicitly define unknown handling: absence of observations does not permit motion.

Task 1. Dijkstra and A* on a robot map

Create an open field, a U-shaped trap and two corridors with different costs. Implement a common `solve(start, goal, map)` interface, path reconstruction and an expanded-vertex counter. Output: path, cost, runtime, expansions and `no_path` status. Test `start=goal`, a goal inside an obstacle, an unreachable goal and equal A*/Dijkstra cost with admissible h . Use BFS as a reference only for equal edge costs.

Task 2. A narrow passage and a dynamic obstacle

Use a circular body model and a passage around a box. Build two costmaps for different footprint sizes and compare passage availability. After planning, add an obstacle in the middle of the route. Mark the old path stale, stop the simulated execution process and replan. Save map and plan versions, the plan-invalidation log, and the new path or a correct rejection.

Check every segment against the current map; a diagonal must not pass between touching corners of two occupied cells. Save a case where the body centre passes but the footprint hits a wall. Explain the future distinction for a leg: the body-centre route does not define a contact sequence, and height variation requires another state model. In M16, demonstrate this boundary and do not call a 2D path a complete walking plan.

Sampling-based search and the manipulator scene

Configuration space

Configuration q contains joint angles, body position or other system coordinates. A workspace obstacle becomes a set of forbidden q values. Checking one endpoint does not establish the validity of motion between points. Study joint limits, self-collision and collisions with the environment and attached object. The configuration-space (C-space) metric must account for units and periodic angles.

PRM builds a configuration network for repeated queries; RRT grows a tree; RRT* improves connections to reduce cost as sampling grows. Probabilistic completeness and asymptotic optimality hold under assumptions and a limiting budget; they do not guarantee a solution within two seconds. A narrow passage may rarely be sampled. Post-search path simplification also changes the result and requires checking.

Task 3. Compare sampling-based search

Start with a 2D point or disc model among rectangular obstacles in OMPL. Compare RRTConnect with RRT* or PRM using equal time per query: at least 10 random-number seeds for three scenes. Save the exact-solution fraction, time to first solution, length and minimum clearance; account for PRM construction cost separately. Prepare configs, seed-list and a table of every run, including timeouts. A missing solution must not be counted as a zero-length path.

Task 4. A MoveIt planning scene

Use a ready-made arm, table and box. Specify object dimensions, positions and orientations in one coordinate frame; specify start and goal, and plan an approach in simulation. Then attach the object to the gripper and replan. Save the scene, a valid path and a failure example. Check that the attached object participates in collision checks and that its frame and scale are correct.

Separately check the path with sufficiently fine discretisation and compare with a coarse check: a thin obstacle between waypoints must be detected. Document the limits of discrete checking; without continuous collision checking, it is not a mathematical proof of no contact along the entire continuous path. The future arm uses this setup to approach an object, while a future leg uses similar checks for link motion.

Trajectories: time, smoothness and actuator limits

Core topics

Study linear interpolation, cubic and quintic profiles, and continuity of position, velocity and acceleration. Jerk is the derivative of acceleration, in rad/s^3 for a revolute joint. A geometric spline may be smooth in parameter s but become infeasible after incorrect time parameterisation. Even within velocity limits, a trajectory may require excessive torque; distinguish dynamic constraints explicitly.

For a simple motion from q_0 to q_1 , use $q(t)=q_0+(q_1-q_0)\cdot s(t/T)$, where $s(\tau)=10\tau^3-15\tau^4+6\tau^5$, $0\leq\tau\leq1$. Velocity and acceleration are zero at both endpoints. Study the derivatives: velocity scales as $1/T$, acceleration as $1/T^2$ and jerk as $1/T^3$. This gives an understandable way to slow an educational movement, but does not automatically check collisions or dynamics.

Task 5. A leg or finger joint profile

Specify q_0, q_1 and teaching limits in the configuration, explicitly labelling them as simulation parameters. Calculate the profile for three T values, plot q, v, a, j and find the smallest tested T that satisfies the limits with a margin. Output: trajectory.csv, plots, assumptions and a validator. Test $q_0=q_1$, $T\leq0$, non-numeric inputs and an excessively short movement. For a real actuator, obtain limits from specifications and load tests.

Task 6. Check a path after smoothing

Take the Task 4 path, apply time parameterisation with an available MoveIt tool, and check the complete result: joint limits, strictly increasing times, derivatives and collisions. Establish whether the chosen library changes geometry during processing. Output: original and processed paths, times and rejection reasons. Construct a corner near an obstacle where smooth interpolation intersects it.

For an introduction to trajectory optimisation, focus on the formulation: variables are states and controls; constraints are dynamics and obstacles; the objective is time, energy or smoothness. Solve a small scalar example or study a worked solution without building a full optimiser. Selecting path topology and continuously optimising a trajectory solve different parts of the overall task; both outputs require checking.

Autonomous behaviour: states and recovery

Minimal task logic

Define idle, planning, executing, waiting, recovering, failed and completed states. For each, specify inputs, actions, exit conditions and permitted cancellation. An FSM supports explicit transition accounting; a behaviour tree supports reusable conditions and actions returning SUCCESS, FAILURE and RUNNING. Study Sequence, Fallback, reactive condition checks and interruption of an asynchronous action.

Recovery needs a cause and an attempt limit. For a stale map, refresh observations and replan; endless retries do not make an unreachable goal reachable. Cancelling an active ROS action requires acknowledgement and an unambiguous state, otherwise motion may continue after the goal changes. Measure local timeouts with a monotonic timer; describe simulation time and recording replay separately.

Task 7. A behaviour tree for a route

Build "check localisation → plan path → follow path", with a branch that waits for fresh data and bounded replanning. First implement deterministic action stubs, then connect a Nav2 action in simulation. Output: BT XML or code, a transition table and an event log. Test planner FAILURE, endless RUNNING and user cancellation; configuration must bound the number of attempts.

Task 8. Measure recovery time

Block the path in simulation, then clear it; measure detection time, time to a new plan and time until motion resumes. For a permanent obstacle, expect failed or waiting for the operator, rather than a false completed. Output: a recording, BT settings, events and two plots. Do not transfer commands from the wheeled Nav2 example directly to a walking body: it needs a separate executor with contact checks.

MDP and POMDP through a concrete example

Describe a choice between a short unknown section, a detour and an inspection using state, action, transition and reward. In a POMDP, the true state is hidden; decisions use a belief distribution updated from observations. Tabulate two traversability hypotheses and update their probabilities after a camera observation. Reinforcement learning is not required here. Account for map uncertainty before permitting motion.

Integration setup: plans have a validity period

Request and plan requirements

Include request_id, current starting position and orientation, goal, frame_id, map_version, robot model and limits in the request. The response contains exact/approximate/no_path/timeout status, the path, its validity conditions and checking parameters. approximate status does not establish goal attainment. Before accepting a trajectory, the executor checks map version, localisation availability and whether the initial state is current.

Task 9. Connect search, trajectory and behaviour

1. Validate inputs. 2. Calculate an A* or Movelt path for your setup. 3. Independently check collisions. 4. Set motion timing and check limits. 5. Send the trajectory to the simulation executor. 6. Monitor map and status changes. 7. On a violation, block acceptance of the next segment, cancel the action and move the BT into recovery. 8. Save the final state and its cause.

For the body route, use a path between two map points. For the arm, approach a pregrasp pose and return to the initial state; the grasp cycle itself belongs to M17. A common validator checks shared data requirements, but the setups have different geometry. Do not present the same point list as a universal executor for wheels, legs and manipulators.

Task 10. Reproduce an asynchronous fault

Create this situation: plan A is delayed, the user has already requested B, and the old successful response A then arrives. The executor must reject A using request_id and version, and the log must show why. Repeat with delayed cancel acknowledgement and completion of the old action. Output: a reproducible GUI-free test and an event sequence. This is an asynchronous race in robotics: a mathematically correct path may belong to a task that is no longer current.

Final package contents

Save comparative tests under identical conditions with every seed, Movelt and Nav2 scenes, path and trajectory checks, the tree and FSM table, eight failure scenarios, and a README with commands, versions and limitations. Two short videos must show normal execution and correct recovery. Measure time from event logs rather than edited video. This software setup does not validate the physical safety of an actuator or real workspace.

Scenarios the planner must handle

Geometry and data

S1 - start or goal inside an obstacle: return `invalid_request` before search and do not begin execution. S2 - A passage narrower than the body footprint: a point path may exist, but reject the body plan. Check the narrowest point by hand. S3 - A thin wall between waypoints: independent collision checking must find the intersection; compare two checking resolutions.

S4 - An unknown strip across the map: forbid mode produces a detour or failure; allow is permitted only as an explicitly labelled teaching experiment. Save both settings without silently changing them after failure. S5 - A new obstacle during motion: cancel the old plan, increment version and use the current scene for the new plan. Measure detection time and record the FSM response.

Timing and logic

S6 - An infeasible trajectory: shorten its duration until velocity or acceleration limits are violated. The check must reject the result and identify the segment and measured quantity. Separately add non-monotonic timestamps. S7 - Timeout and a stuck action: a stub returns `RUNNING` beyond the limit. The tree must bound recovery attempts and finish with an understandable status. Record attempts and waiting time even if a path is eventually found.

S8 - Cancellation and a late response: request B replaces A, then result A arrives. No segment of A may be accepted after the change of current request is confirmed. Also test M15 localisation loss: block new global goals, and do not mark an ended action successful merely because updates stopped.

Metrics and comparison rules

For search, measure expansions, the number of expanded vertices; cost, the sum of specified edge costs; and runtime, the duration of the selected code segment. For sampling methods, measure exact-solution fraction, time to first solution, final cost, minimum clearance and number of collision checks. Report timeout separately; show median time for successful runs together with success fraction.

For the autonomous task, measure success fraction, recovery count, time to detect a violation and build a new plan, and rejected stale-command count. State clocks, tick frequency and hardware. Body-path length is the sum of segment lengths in metres. For a joint path, the sum of angular-increment norms means something different: specify joint weights and do not compare it with metres. Minimum clearance is calculated between geometric bodies at checked configurations; state discretisation and units. A map may penalise wall proximity, so the shortest path need not have the lowest cost. No collisions in a finite simulation set describes that set; it does not prove their absence for every input, parameter and physical contact.

Sessions 1-40: graphs and configuration space

Hours 1-10: five 2-hour sessions

1-2: distinguish paths, trajectories and task decisions; draw input/output diagrams. 3-4: BFS/Dijkstra/A*, queues and cost; expand a small map by hand. 5-6: heuristic admissibility, consistency and A* termination. 7-8: body footprint, obstacle inflation, unknown and costmap units. 9-10: comparison protocol, dynamic obstacle and version checks. Use your algorithm-problem experience, adding real robot size and the consequences of unknown cells.

Hours 11-20: practical 1

11-12: Task 1, common interface and Dijkstra. 13-14: A*, optimal-cost checks and BFS for unit-cost edges. 15-16: three maps, metrics and expanded-vertex display. 17-18: Task 2, different footprints and a narrow passage. 19-20: map changes, invalidation of the old plan and reproducible no_path. By the end of the block, prepare a baseline for comparison with subsequent algorithms.

Hours 21-30: five 2-hour sessions

21-22: configuration space of a two-joint arm, self-collisions and limits. 23-24: PRM/RRT/RRT*, metric, sampling and time limits. 25-26: probabilistic properties and narrow passages; record conditions that cannot be attained in finite time. 27-28: the MoveIt scene, attached objects and collision matrix. 29-30: independent path checking and motion discretisation.

Hours 31-40: practical 2

31-32: Task 3, OMPL and scenes for two planners. 33-34: 10 seeds, timeouts and a complete comparison. 35-36: Task 4, a ready-made arm, table and box. 37-38: attached-object movement, frame and size checks. 39-40: a thin obstacle, an unreachable goal and a report on sampling-search limits. Keep total time equal and account separately for PRM preprocessing.

If the MoveIt GUI does not start, record environment limitations and try opening the prepared scene without a graphical interface within the allocated time. Do not spend the entire practical on a different operating system. If necessary, simplify the arm while retaining joint limits, an obstacle and attached-object checks. Analysing a specific collision-checking error is more useful than adding models.

Sessions 41-75: trajectories and the autonomous task

Hours 41-50: theory in 2-hour sessions

41-42: $q(t)$ derivatives, units, linear and cubic interpolation. 43-44: quintic profiles and T scaling. 45-46: velocity, acceleration, jerk, boundary conditions and actuator limits. 47-48: MoveIt time parameterisation and geometry rechecking. 49-50: trajectory-optimisation foundations, objective and constraints; comparison with path-topology search.

Hours 51-60: practical 3

51-52: Task 5, one-joint profile and three durations. 53-54: analytical and numerical derivatives, validator and edge-case inputs. 55-56: Task 6, time parameterisation of an arm path. 57-58: smoothing near an obstacle and independent checking. 59-60: trajectory and limit files, plus rejection reasons; prepare the simulation-executor interface.

Hours 61-65: theory 2+2+1

61-62: FSM and BT, RUNNING/SUCCESS/FAILURE, Sequence/Fallback. 63-64: cancellation, timeout, recovery and late responses; draw the Task 10 event sequence. 65: MDP/POMDP through detour, inspection or motion choices and a traversability belief distribution. These hours do not include an additional RL course: understand which information is unavailable to simple logic.

Hours 66-75: practical 4

66-67: Task 7, a tree with deterministic action stubs and transition checks. 68-69: Task 8, Nav2 simulation with temporary and permanent obstacles. 70-71: Task 9, requirements for handing off plans, trajectories and execution, and map age. 72-73: Task 10, S1-S8, cancellation and late responses. 74-75: reproduction from a clean start, comparison table and final report. Every task fits within the 75 hours.

What to cut first

First omit a third planner, an extended smoothing-method comparison and additional interfaces. Then reduce scenes to three and shorten simulations while retaining every failure type. Dijkstra/A*, one sampling experiment, a MoveIt collision scene, trajectory limits and BT recovery timing remain mandatory. Keep stale-plan checks: an asynchronous race can negate the correctness of the search algorithm.

Acceptance: correct rejection is also a result

Required verification results

Compare at least two planners under one protocol; verify Dijkstra/A* cost on reference graphs. In the sampling experiment, use multiple seeds, count timeouts and check the resulting path. The MoveIt scene must include an obstacle and an attached object. Reject an infeasible trajectory with a specific reason; after smoothing, recheck collisions and limits.

The behaviour tree completes the task in a normal scene, recovers from a temporary obstacle and ends its attempts when the goal remains unreachable. Cancellation works during RUNNING, and a late result does not replace the current task. Logs connect map_version, request_id, plan and execution. All eight scenarios are reproducible; launch instructions do not depend on the author's manual GUI actions.

Self-check

1. When do BFS and Dijkstra produce the same cost? 2. How does an incorrect h remove A*'s expected optimality? 3. Why does footprint size affect whether a path can be found? 4. How does PRM usage differ from RRT? 5. Why does RRT* not promise an optimal path in fixed time? 6. What does checking only waypoints miss?
7. How do v, a, j change when T doubles? 8. Why can a smooth spline collide with a wall? 9. How do velocity constraints differ from torque constraints? 10. Why is RUNNING neither success nor failure? 11. How do you cancel an old task and reject its late response? 12. What additional information does a POMDP require compared with a fully observed state?

Handover to manipulation and legs

Pass the arm scene, collision objects, pregrasp path, verified executor, limits and event log to M17. For later walking, retain the map with unknown, footprint assumptions and constraints absent from the 2D route. Contact selection, stability, foot reachability and body dynamics will require a new model. Cancellation and task-freshness checks remain applicable.

The final demonstration must include no_path, an excessively fast trajectory and recovery, as well as a successful route. Explain the precise level of assurance your test suite provides. "No valid plan found" is more useful than moving on an uncertain result. Do not claim physical safety of a real system based on one month of simulation.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

S. LaValle, Planning Algorithms (2006). [Author's online book](#). Ch. 2 Discrete Planning - graph search; Ch. 4 The Configuration Space - C-space obstacles; Ch. 5 Sampling-Based Motion Planning - PRM/RRT. Before Tasks 1-4, write out the formulation, validity criterion and termination condition.

K. Lynch, F. Park, Modern Robotics (2017). [Book and learning resources](#). §9.1-9.2 Point-to-Point Trajectories, §9.3 Polynomial Via Point Trajectories, §9.4 Time-Optimal Time Scaling. In Tasks 5-6, distinguish $q(s)$ and $q(t)$, calculate velocity and acceleration, and check limits.

Task-management reference reading

M. Colledanchise, P. Ögren, Behavior Trees in Robotics and AI: An Introduction. [Authors' book version](#). §1.3 Classical Formulation of BTs, §2.1 Finite State Machines; in Ch. 3 Design principles, §3.1-3.2 on success conditions and reactivity. For Tasks 7-8, draw a tree for planning, following and recovery; check RUNNING returns, stopping and condition reevaluation.

Three implementation references

[OMPL: How to Benchmark Planners](#) - Writing benchmarking code and Processing the benchmark log. In Task 3, save exact/approximate, timeout, run count and validity of the solution found.

[MoveIt: Planning Scene](#) - Collision checking, Constraints, State validity. In Task 4, include self-collision and environment collisions; check a changed scene before execution.

[Nav2: Detailed Behavior Tree Walkthrough](#) - Navigation Subtree and Recovery Subtree. For Tasks 7-8: a minimal planning, following and recovery sequence; check XML and plugins against Jazzy.

MDP/POMDP are currently studied only through the distinction between state and observation. After reading, explain why an exact path in a model can be unexecutable and why timeout failure must remain visible in comparative test results.

Terms and definitions

Configuration q . A set of independent coordinates defining the robot's geometric arrangement; velocities usually belong to an expanded state.

Configuration space (C-space). The space of all robot configurations; collisions between geometric bodies define a forbidden region in this space.

Footprint. The planar projection of the area occupied by the robot, used in mobile-base planning and passage checks.

Costmap. A discrete representation of obstacles and movement penalties; a cell's cost need not be its occupancy probability.

Geometric path $q(s)$. A configuration sequence parameterised by progress s ; time, velocity and acceleration are not yet specified.

Trajectory $q(t)$. Motion specified over time, allowing calculation of velocities, accelerations and actuator requirements.

Dijkstra. Shortest-path search with nonnegative edge costs, processing the smallest current cost successively.

A*. Search selecting a node by the sum of accumulated cost g and the estimated remaining cost h .

Admissible heuristic. A remaining-cost estimate that never exceeds the true optimum; search guarantees also depend on implementation and graph conditions.

PRM. A roadmap method using valid random configurations and checked connections between neighbouring samples.

RRT. A search tree extended towards samples in the space; the basic algorithm aims to find a path rather than optimise it.

Probabilistic completeness. A property in which the probability of finding an existing solution approaches one as resources increase and assumptions hold.

Collision checking. Checking robot geometry for intersection with itself and the world; sparse point checks may miss collisions between them.

State validity. Satisfaction of specified configuration, scene and task constraints; it does not establish validity of an entire connecting trajectory.

Time parameterisation. Assignment of path progress as a function of time, accounting for velocity, acceleration and other constraints.

Jerk. The time derivative of acceleration, expressed in m/s^3 or rad/s^3 ; describes how abruptly the applied motion changes.

Behaviour tree (BT). A hierarchical structure executing actions and conditions that return success, failure or ongoing-execution status.

Tick. A signal to evaluate a tree node; tick frequency determines the timeliness of condition checking and action cancellation.

FSM. A finite state machine with explicit states and transitions driven by events or conditions; defines permissible behaviour order.

Recovery. A bounded procedure following a detected failure, with a completion condition, retry limit and a route to safe stopping.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, UTM with Ubuntu 24.04 ARM64, ROS 2 Jazzy, Gazebo Harmonic, and M14-M15 maps, scenes and recordings. The two teaching setups require simulated mobile-base and manipulator models. No new hardware purchases are mandatory; do not order a physical arm, wheeled base or additional actuators.

Install now

In the existing Ubuntu guest, install Nav2 navigation2/nav2_bringup, MoveIt 2 and OMPL with dependencies for the Jazzy version in use. MoveIt: Binary Install specifies ros-jazzy-moveit for Ubuntu 24.04; the official Nav2 repository is the starting point for package installation and compatible bringup. Record package versions and arm64 availability; do not copy Rolling commands without checking.

Install MoveIt 2 in M16 because the manipulator-scene laboratory runs here; reuse the stack in M17. Initially use OMPL from MoveIt dependencies, adding a separate C++ benchmark build only for metrics required by the task. Python bindings are not mandatory dependencies for your own A*/Dijkstra implementation.

Use a BehaviorTree.CPP version compatible with Nav2; Reactive and Asynchronous behaviors explains Sequence/ReactiveSequence and halt. A BT GUI editor is optional. Retain one RMW, the RViz configuration and prepared demo packages; do not update the entire workspace merely for a new documentation page.

Check the installation

1) In Nav2, send a reachable goal on a small map, then an unreachable one; check the action result and cancellation. 2) In MoveIt, load a prepared model, table and box; obtain a path and reject a collision. 3) Check availability of the chosen OMPL planners, exact/approximate statuses and timeout. 4) Record RUNNING, tree cancellation and recovery, and final cessation of the command.

Test RViz and Gazebo in UTM with a separate small run: an installed package does not establish working graphics. If problems occur, save console logs and paths for analysis; visual verification remains open until the output is actually viewed. These simulation tasks do not need USB, but reused physical peripherals still require their previous checks.

Reading and setup fit within the 75 hours. An onboard Pi/x86, GPU and physical arm kit belong to later conditional stages; after M17, the separate OPT120 has its own CAD/BOM and single-finger validation.

Motion fundamentals: timing along a path

Required: why geometry is insufficient

The same path can be traversed slowly or abruptly. Configuration geometry determines collisions, while velocity, acceleration and required torques also depend on time. Prerequisite: take one revolute leg or finger joint, with q in radians, and explain $\dot{q}$ and $\ddot{q}$ in rad/s and rad/s².

Two functions rather than one point list

A path $q(s)$, $0 \leq s \leq 1$, specifies configuration order; a monotonic $s(t)$ specifies progress. The chain rule gives $\dot{q} = q'(s)\dot{s}$ and $\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}$. The first acceleration term comes from path curvature and the second from changes in the rate of progress. Thus smoothness of $q(s)$ does not replace checks of time derivatives.

For a straight joint-space path $q(s) = q_0 + \Delta q \cdot s$, choose $s(t) = 3\xi^2 - 2\xi^3$, $\xi = t/T$. Then $\dot{q} = (\Delta q/T)(6\xi - 6\xi^2)$, $\ddot{q} = (\Delta q/T^2)(6 - 12\xi)$. Velocity is zero at both endpoints; acceleration there is nonzero and jumps when transitioning to rest. The quintic profile in the existing Task 5 adds zero boundary accelerations.

Numerical example and duration selection

Let $q_0 = 0$, $\Delta q = 0.6$ rad and $T = 2$ s. Maximum velocity at $t = 1$ s is $1.5\Delta q/T = 0.45$ rad/s. Maximum $|\ddot{q}|$ at the endpoints is $6\Delta q/T^2 = 0.90$ rad/s². With example limits of 0.30 rad/s and 0.60 rad/s², this profile is too fast.

The velocity limit gives $T \geq 1.5 \cdot 0.6 / 0.30 = 3$ s; the acceleration limit gives $T \geq \sqrt{(6 \cdot 0.6 / 0.60)} = \sqrt{6} \approx 2.45$ s. Thus this cubic profile requires $T \geq 3$ s. At $T = 3$ s, the maxima are 0.30 rad/s and 0.40 rad/s²; choose a larger T for a margin. This result applies to the chosen profile shape, not the globally shortest time for any motion.

Foot velocity equals $J_p(q)\dot{q}$ and varies with configuration; one joint limit does not specify constant foot speed. Moreover, the torque needed to support weight does not disappear with slow motion. After checking velocity and acceleration constraints, check dynamics, contacts and geometry; calculating torque does not set the control mode of a position-controlled servo.

Two exercises in Tasks 5-6

1. Save $q, \dot{q}, \ddot{q}$ for $T = 2, 3, 4$ s in trajectory.csv. Compare analytical extrema with a dense grid and numerical derivatives; check $q(0), q(T)$, zero velocities and rejection of $T \leq 0$. Then use the same validator on the Task 5 quintic profile: its maximum coefficients differ.

2. Take the arm path near the box: a monotonic change of time parameterisation alone preserves path geometry, while waypoint-based path smoothing may change it. Show both versions and repeated collision checking; include one correct rejection and a plot of the violated constraint.

Time accounting: 1 hour from 41-42 for the derivation; 1 hour each from 51-52 and 53-54 for the numerical example and validator. Geometry checking remains within the existing 57-58. The budget stays at 75 hours, with no additional setup required.

Assigned reading: Lynch/Park, [Modern Robotics §9.1-9.2, part 1](#) - path and time; [part 2](#) - cubic and quintic time scaling and boundary conditions.